Poool Game: Concurrent Implementation Report

Assignment 01 - PCD Course

Academic Year 2025-2026

Abstract

This report presents a concurrent implementation of the Poool Game, a 2D board game where two players (human and bot) control balls to push smaller balls into holes. The document provides a detailed analysis of concurrency challenges, design architecture, behavioral specifications using Petri Nets, performance evaluation, and formal verification using JPF (Java Path Finder). The concurrent implementation demonstrates significant improvements over a sequential baseline while effectively exploiting available processor cores through multithreading and task-based approaches.

Contents

1 Problem Analysis

1.1 Problem Description

The Poool Game is a two-player billiard-like game on a 2D board with the following characteristics:

- **Dynamic Objects**: Multiple balls (typically ranging from 2 to 4500) moving and interacting on the board
- **Physics Simulation**: Elastic collisions, friction forces, and momentum transfer between bodies
- **Players**: Two players (human controlled via keyboard, bot controlled autonomously) with scoring systems
- **Game Objective**: Insert small balls into holes at board corners; winner is determined by score or by eliminating opponent's ball

1.2 Multi-Threaded Related Challenges

Introducing parallelism into the physics pipeline exposed several concurrency challenges that influenced the overall architecture of the system. This section analyses each challenge and motivates the design choices.

1.2.1 Collision Detection Complexity

The most computationally demanding operation in the simulation is ball-ball collision detection. A naive approach tests every ball against every other ball each tick, yielding a time complexity of $O(n^2)$ where n is the number of small balls on the field. While parallelising this work across N workers reduces the per-thread cost to $O(n^2/N)$, the quadratic growth dominates: with a large number of balls, even distributing the work across all available cores is insufficient to maintain interactive framerates, because the total number of pair checks grows much faster than the number of available threads.

The underlying observation is that the vast majority of these checks are geometrically impossible: two balls of radius r can only collide if their centres are within distance $2r$ of each other. Checking distant pairs is wasted work. A spatial hashing grid — partitioning the field into cells of side $2r$ and testing each ball only against the balls in its 3×3 cell neighbourhood — reduces the average complexity to $O(n)$, since the number of neighbours per cell is bounded by a constant determined by ball density and radius. This algorithmic improvement is a prerequisite for parallelism to be effective at scale: no amount of additional threads can compensate for a fundamentally super-linear algorithm when n is large.

1.2.2 Memory Bandwidth Saturation

Distributing ball updates across multiple worker threads introduces the risk of memory bandwidth saturation and cache thrashing. If balls are assigned to workers randomly, the working set of each thread is scattered across the heap, causing frequent cache misses as each core loads and evicts cache lines that other cores also need. In the worst case, the

overhead of cache invalidation traffic between cores can exceed the computational benefit of parallelism.

A related issue arises from shared mutable state: the `lastTouchedBy` map, which records the last player to touch each ball for scoring purposes, is accessed by both the worker threads (to reset entries after ball-ball collisions) and the update thread (to update entries after player-ball collisions). Protecting this map with a single global lock on `Board` would serialise all workers on every collision, effectively eliminating concurrency for any tick where collisions are frequent. Furthermore, acquiring `synchronized(board)` while already holding `synchronized(ball)` introduces a lock ordering that can interleave incorrectly with other synchronized methods on `Board`, creating a potential deadlock.

Both issues are addressed by two targeted measures: assigning each worker a *contiguous* partition of the ball list (improving spatial locality and cache reuse), and replacing the `HashMap` with a `ConcurrentHashMap` (enabling concurrent updates without a global lock and eliminating the nested lock dependency).

1.2.3 Synchronization Requirements

Parallelising the physics pipeline requires coordinating multiple threads across several phases per tick, each with fundamentally different synchronization semantics. Three distinct requirements were identified.

Phase boundary enforcement. Position updates must be fully completed by all workers before any collision resolution begins. This is an *all-to-all* dependency: every thread must complete its current phase before any thread may advance to the next. The natural primitive is a `CyclicBarrier` — a reusable synchronization point that blocks all $N + 1$ parties (workers plus the `UpdateThread`) until the last one arrives, then releases all of them simultaneously. Its cyclic nature makes it well suited to the repeating tick structure without requiring reallocation.

Deadlock-free per-object locking. Concurrent collision resolution requires each worker to acquire locks on two `Ball` objects simultaneously. If different workers acquire these locks in inconsistent orders, circular waits can form — the classical lock inversion problem. The solution is a canonical lock-ordering protocol: locks are always acquired in strictly ascending index order (the position of each ball in the shared snapshot), which is a strict total order that prevents any circular dependency by construction.

Asymmetric synchronization for grid construction. The spatial grid is rebuilt each tick by a single designated thread (Worker-0), while all other threads must wait for it to be ready before entering the collision phase. This is a *one-to-all* dependency — one producer unblocks N consumers — which is semantically distinct from the all-to-all synchronization of a `CyclicBarrier`. Using a barrier here would be technically correct but conceptually misleading, as it implies parallel work where none exists. The appropriate primitive is a custom `ReusableLatch`: Worker-0 calls `open()` after `grid.build()`, all other threads call `await()` and block until the latch opens. The latch is reset by the `GameController` at the beginning of each tick, before any worker is activated, making it safely reusable across ticks without reallocation.

1.3 Task-based Related Challenges

The task-based concurrent implementation using the Executor Framework introduces a different set of challenges compared to the thread-per-entity model. The Poool imple-

mentation uses work-stealing task decomposition via `WorkerPool` to parallelize physics updates and collision detection.

1.3.1 Computational Complexity

Task-based parallelism in the Poool implementation must address several complexity-related challenges:

- **Task Granularity Trade-off**:
  - Fine-grained tasks (small chunk size) improve load balancing but increase task creation and scheduling overhead
  - In `WorkerPool`, each task updates balls in range $[i, i+\text{chunkSize})$, where chunk size is computed as $\text{chunkSize} = \max(1, n/k)$ with $k = 2 \cdot \text{numProcessors}$
- **Collision Detection Complexity**:
  - `CollisionTask` must verify all pairs (i, j) with $i < j$ without duplication
  - Naive task decomposition (dividing range $[0, n)$ into contiguous chunks) would create a *triangular* workload: the task covering the lowest indices compares its balls against almost the entire list, while the task covering the highest indices does almost no work
  - Current implementation instead partitions indices *by stride*: task k (of $w = \min(n_{\text{workers}}, n)$ tasks) processes indices $k, k+w, k+2w, \ldots$, so each task receives an interleaved mix of "heavy" (low-index) and "light" (high-index) balls, balancing the total number of comparisons across tasks
  - Total complexity remains $O(n^2)$ across all tasks, but the per-task share is now approximately $O(n^2/w)$ instead of being skewed toward the first task
- **Memory Bandwidth Saturation**:
  - Task-based approach requires all workers to read shared Ball objects' position/velocity data
  - With k cores simultaneously accessing the same memory region, cache line contention increases
  - ConcurrentHashMap operations on `lastTouchedBy` introduce additional memory traffic
- **Synchronization Overhead in Collision Detection**:

```
1    synchronized (first) {
2        synchronized (second) {
3            // Atomic collision resolution
4            Ball.resolveCollision(a, b);
5        }
6    }
```

Listing 1: Ordered Lock Acquisition

- Each collision check acquires locks on two Ball objects using `synchronized(first)` then `synchronized(second)`
- Lock ordering (by `System.identityHashCode()`) prevents deadlock but adds comparison overhead
- If two tasks try to resolve the same collision pair with different order, one will block the other

1.3.2 Temporal Constraints

Task-based parallelism introduces timing-related constraints that differ from the thread-per-entity model:

- **Barrier Synchronization Latency**:
  - In `WorkerPool`, `PlayerCollisionTask`, `parallelBallUpdate()` and `parallelCollisionDe` use `invokeAll()`, which is a synchronization barrier
  - All tasks must complete before control returns to the GameLoop
  - Frame time = max(task duration) + overhead
- **Task Queue Delay**:
  - Created tasks are submitted to the ExecutorService queue
  - If all workers are busy, new tasks wait in queue, delaying their execution
  - With a fixed thread pool of size k, tasks may experience queueing delay proportional to queue depth
  - This adds non-deterministic latency to each game frame
- **Future.get() Blocking Behavior**:
  - The main GameLoop thread calls `f.get()` (code in class `WorkerPool`) on all futures from `invokeAll()`
  - This is a blocking operation—the GameLoop cannot proceed until *all* tasks complete
  - Any uncaught exception in a task will be re-thrown at `get()`, potentially crashing the loop

1.3.3 Synchronization Requirements

Task-based parallelism requires careful synchronization to ensure memory consistency and avoid data races:

- **ConcurrentHashMap for lastTouchedBy**:
  - Multiple `CollisionTask` instances concurrently update `lastTouchedBy` map via `put()` calls
  - Using a non-concurrent map would cause data races
  - ConcurrentHashMap uses bucket-level locking (segment locking), reducing contention vs. a single global lock

  - However, repeated `put()` calls on the same Ball key still incur synchronization overhead

- **Read-Write Visibility**:
  - After `invokeAll().get()`, all field writes in tasks are visible to the main GameLoop (happens-before guarantee)
  - The Java Memory Model ensures that synchronization actions (lock acquire/release) create memory barriers
  - Without these barriers, updates to Ball.vel and Ball.pos made by one task might not be visible to the next phase

- **Task Isolation**:
  - `BallUpdateTask` accesses `board` (the Board object) to query boundary constraints
  - This creates a shared dependency: if Board is modified during task execution, physics updates may read stale boundary data
  - Current implementation assumes Board state (bounds, player positions) is immutable during a phase
  - If asynchronous modifications occur (e.g., player ball movement), visibility must be explicitly synchronized

- **Exception Handling and Task Failure**:
  - If a task throws an unchecked exception, `f.get()` will re-throw it via `ExecutionException`
  - The GameLoop must handle `InterruptedException` (task thread interrupted) and `ExecutionException` (task computation failed)
  - Without proper handling, the entire game loop may crash
  - Current implementation in `GameLoop.run()` catches both exceptions and breaks the loop gracefully

2 Design and Architecture

2.1 Architectural Overview for Multithreading

The project is structured following the **Model-View-Controller (MVC)** pattern, with a strict separation of responsibilities across three packages: `pcd.assignment01.model`, `pcd.assignment01.view`, and `pcd.assignment01.controller`.

- **Model**: The model package contains all domain state and physics logic, with no knowledge of rendering or threading policy. The central class is `Board`, which owns the list of `Ball` objects, the two player balls, the board boundaries, the scores, and the game-over flag.
  `updatePlayers()`, `resolveBallCollision()`, `resolvePlayerCollisions()`, and `checkHolesAndGameOver()` are the only methods through which the controller modifies the model state.

- **View**: The view package contains `View`, `ViewFrame`, and `ViewModel`. The view never reads directly from `Board`: it reads exclusively from `ViewModel`, a thread-safe snapshot object whose fields are updated by the controller once per tick via `update(Board, int)`. This decoupling ensures that the Swing Event Dispatch Thread (EDT) always paints a consistent, fully-populated snapshot without ever competing with the physics threads for `Board`'s intrinsic lock.

  `ViewModel` stores `BallViewInfo` records (position and radius) for all small balls and for each player, the current scores, the FPS counter, and the game-over message. The game-over message is written only once, through `setGameOverMessage(String)`, called by the controller at the end of the game; it is never written during normal tick updates, avoiding a race betweenphysics threads and the EDT.

  `RenderSynch` provides a generation counter that lets the update thread block until the EDT has completed the current repaint, preventing frame tearing without busy-waiting.

- **Controller**: The controller package contains `GameController`, `BotController`, `InputHandler`, and `WorkerThread`.

  `GameController` is the orchestrator: it owns the update loop thread, the worker pool, both `CyclicBarrier` instances, and the `BotController`. It is the only component that calls physics methods on `Board` in the correct sequential order after the parallel phases have completed.

  `BotController` encapsulates the autonomous player (player 2) logic in its own thread, waiting on a private monitor until the game physics signal that the ball is still enough to act.

  `InputHandler` translates keyboard events from the EDT into `board.kickPlayer1()` calls, with an early-exit guard on `isGameOver()`. It is registered on the `View` inside `SwingUtilities.invokeLater`, ensuring registration happens on the EDT.

2.1.1 Thread Model

The application runs five categories of threads simultaneously during a game:

1. **Main thread**: bootstraps the model and schedules the Swing initialization on the EDT via `SwingUtilities.invokeLater`. It exits immediately after.

2. **Event Dispatch Thread (EDT)**: It handles key events (forwarded by the Input Handler) and executes repaints triggered by `panel.repaint()`. It never touches `Board` directly: key events call `board.kickPlayer1()`, which is `synchronized`, and repaints read only from `ViewModel`.

3. **UpdateThread**: a single daemon thread owned by `GameController`. It drives the game loop: it advances the physics pipeline each tick, signals `BotController`, copies state into `ViewModel`, and calls `view.render()`. It also participates in both `CyclicBarrier`s as the $(N + 1)$-th party, allowing it to perform the sequential post-barrier phases without additional synchronization.

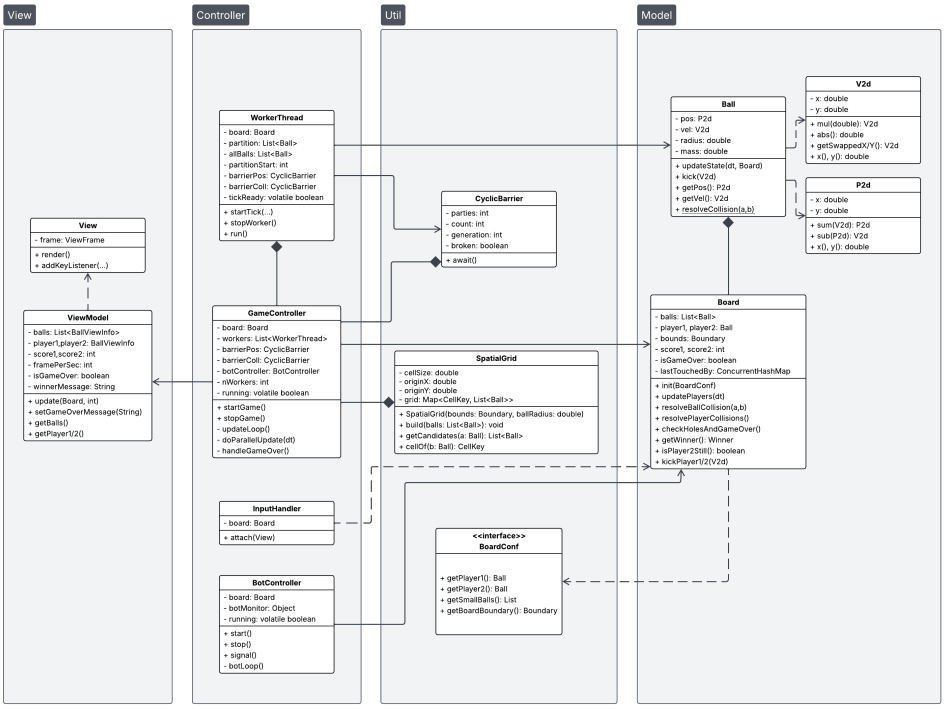

Figure 1: UML with MVC graphics subdivision

4. **WorkerThread-0 ... WorkerThread-N**: a pool that contains exactly N = `availableProcessors()` + 1 persistent daemon threads, created once at startup and never recreated. Each worker waits on its private `tickMonitor` until `GameController` calls `startTick()`, then executes phase 1 (positions) and phase 2 (ball-ball collisions) before waiting again for the next tick.

5. **BotThread**: a single daemon thread owned by `BotController`. It waits on a private `botMonitor` until `GameController` signals that player 2 is still, then applies a random kick impulse subject to a 2-second cooldown.

All threads except the main thread are daemon threads, so the JVM exits cleanly when the window is closed.

2.1.2 Parallel Physics Pipeline

The physics update for each game tick is divided into five phases, two of which execute in parallel across the worker pool.

1. **Player update (sequential)**: Before the parallel work begins, `GameController` calls `board.updatePlayers(dt)`, which advances the positions of the two heavy player balls. This is sequential because (a) there are only two such balls, and (b) the workers' collision phase reads player state indirectly via `resolvePlayerCollisions()`, which runs after the barriers. Keeping player updates sequential avoids any contention.

2. **Position update (parallel)**: The list of small balls is divided into N contiguous, disjoint partitions using a balanced split (each partition differs in size by at most one

element). Each `WorkerThread` receives its partition and calls `ball.updateState(dt)` on every ball it owns. Because the partitions are disjoint, no two workers write to the same `Ball` object during this phase, and no locking is required.

After finishing, each worker calls `barrierPositions.await()`. The `UpdateThread` also calls `barrierPositions.await()` as the $(N+1)$-th party. The barrier scatters all threads into phase 3 only once every worker (and the controller) has finished phase 2, guaranteeing that all positions are fully updated before any collision is resolved.

3. **Spatial grid construction (Worker-0 only)**: After all positions have been updated in phase 2, computing ball-ball collisions naively requires checking all $O(n^2)$ pairs. To reduce this to $O(n)$ on average, the application uses a **spatial hashing grid** implemented in the `SpatialGrid` class (package `pcd.assignment01.util`).

   The field is partitioned into square cells of side $\mathit{cellSize} = 2 \cdot r_{\text{ball}} + \varepsilon$, where $r_{\text{ball}} = 0.01$ is the radius of the small balls. This choice guarantees that two balls whose centres lie in non-adjacent cells have distance $> \mathit{cellSize} > 2 \cdot r_{\text{ball}}$ and therefore cannot overlap: no colliding pair is ever missed by checking only the 3×3 neighbourhood of cells around each ball.

   The grid is stored as a `HashMap<CellKey, List<Ball>>`, where `CellKey` is a value type wrapping the integer column and row indices of a cell. Using a hash map rather than a dense 2D array avoids allocating memory for the large number of empty cells that arise with sparse ball distributions.

   The grid must be rebuilt every tick because ball positions change in phase 2. This work is assigned exclusively to **Worker-0** (`isGridBuilder == true`), which calls `grid.build(allBalls)` after `barrierPositions` has fired.

   The synchronization for this phase uses a `ReusableLatch` rather than a `CyclicBarrier`. The distinction is deliberate: a `CyclicBarrier` models *symmetric* synchronization — N threads performing parallel work and waiting for each other. Grid construction is instead an *asymmetric* one-to-all dependency: one producer (Worker-0) generates a result that the other workers must wait for before entering phase 4.

   Worker-0 calls `gridLatch.open()` after `grid.build()` completes; all other workers call `gridLatch.await()` and block until the latch opens. The `UpdateThread` does not participate in the latch — it proceeds directly from `barrierPositions` to `barrierCollisions`. The `GameController` calls `gridLatch.reset()` at the start of each tick, before activating any worker, making the latch safely reusable across ticks.

   The `open()` call establishes a *happens-before* relationship between Worker-0's writes to the grid and all subsequent reads by the other workers in phase 4, as required by the Java Memory Model.

   The synchronization points per tick are therefore:

$$\underbrace{\texttt{barrierPositions}}_{\text{symmetric}} \rightarrow \underbrace{\texttt{gridLatch}}_{\text{asymmetric}} \rightarrow \underbrace{\texttt{barrierCollisions}}_{\text{symmetric}}$$

   The two `CyclicBarrier` instances are each constructed with *parties* $= N + 1$ (one per worker plus the `UpdateThread`). The `ReusableLatch` has no party count — it

is opened by exactly one thread and awaited by all others independently.

4. **Ball-ball collision resolution with spatial hashing (parallel)**

   Each worker iterates over its partition and, for each ball a, calls `grid.getCandidates(a)` to obtain the balls in the 3×3 cell neighbourhood. This returns on average only a small constant number of candidates (proportional to local ball density) instead of all n balls, reducing the total work across all workers from $O(n^2)$ to $O(n)$ on average.

   Each worker also builds a `Map<Ball, Integer>` index (populated in `startTick()`) that maps every ball to its position in the shared `allBalls` snapshot. This allows $O(1)$ index lookup during phase 4, replacing the earlier $O(n)$ `indexOf()` call that would have negated the benefit of the spatial grid.

   The canonical lock-ordering protocol is preserved: a worker processes the pair (a, b) only if $idx(a) < idx(b)$, ensuring each pair is handled exactly once across all workers. Locks are acquired in ascending index order (`synchronized(a)` then `synchronized(b)`), which eliminates the possibility of deadlock by the same argument as before.

   The performance improvement is significant with dense configurations. With the `MassiveBoardConf` (4 500 balls), the naive $O(n^2)$ approach requires checking approximately 10^7 pairs per tick. With the spatial grid, each ball checks on average fewer than 10 neighbours, reducing the total to roughly 45 000 candidate pairs — a reduction of more than two orders of magnitude.

5. **Player collision resolution (sequential)**: `GameController` calls the method `board.resolvePlayerCollisions()`, which is `synchronized` on `Board`.
   This method resolves collisions between each player and every small ball, and between the two players themselves. It also updates the `lastTouchedBy` map, which records the last player to touch each small ball and is used for scoring. This phase is sequential because both player objects are shared across all potential pairs, making fine-grained locking impractical.

6. **Hole detection and game-over (sequential)**: `GameController` calls `board.checkHolesAndGameOver()`, which is also `synchronized` on `Board`. It checks whether any ball or player has entered a hole, removes sunk balls from the list, increments scores using `lastTouchedBy`, and sets `isGameOver` if appropriate. This phase is sequential because it mutates the shared `balls` list and the scoring state atomically.

2.1.3 Synchronization Mechanisms

- **CyclicBarrier**

  The `CyclicBarrier` (in package `pcd.assignment01.util`) is implemented from scratch using Java's built-in monitor protocol (`synchronized`, `wait()`, `notifyAll()`). It maintains three fields: `parties` (the total number of threads that must call `await()` before the barrier fires), `count` (the number of threads still outstanding), and `generation` (an integer that increments on each firing and is used as the condition variable to distinguish the current cycle from the next one, guarding against spurious wakeups).

When the last thread arrives (`count == 0`), it resets `count` to `parties`, increments `generation`, and calls `notifyAll()`. All waiting threads check `myGeneration == generation` as their loop condition: once the generation advances, they know the barrier has fired and they exit the wait.

The barrier also maintains a `broken` flag. If any waiting thread is interrupted, it sets `broken = true` and calls `notifyAll()`, which causes every other waiting thread to wake up and throw `InterruptedException`. This prevents the pathological scenario where a single interrupted thread leaves `count` permanently non-zero, causing all other threads to block forever. The `broken` flag is cleared when the last thread successfully fires the barrier for the next generation.

Two instances of `CyclicBarrier` are created per game, each with $parties = N + 1$ (one per worker plus the `UpdateThread`): `barrierPositions` separates phase 2 from phase 3, and `barrierCollisions` separates phase 4 from the sequential phases 5 and 6.

- **ReusableLatch**

  The `ReusableLatch` (in package `pcd.assignment01.util`) models *asymmetric* synchronization: one producer thread calls `open()` to signal that a resource is ready, and any number of consumer threads call `await()` and block until the latch is open.

  It is implemented with a single boolean flag `open` and the standard monitor protocol. `await()` loops on `while (!open) { wait(); }`, and `open()` sets the flag and calls `notifyAll()`. The `reset()` method closes the latch again for the next cycle; it is called by the `GameController` at the start of each tick, before any worker is activated, so no thread can be in `await()` at reset time.

  One instance of `ReusableLatch` is created per game: `gridLatch` gates the transition from phase 3 to phase 4. Worker-0 calls `gridLatch.open()` after `grid.build()` completes; all other workers call `gridLatch.await()`. The `UpdateThread` does not participate in the latch — it proceeds directly from `barrierPositions` to `barrierCollisions`, since it has no work to do during phase 5.

  The choice of `ReusableLatch` over a third `CyclicBarrier` is deliberate: a `CyclicBarrier` implies that all parties perform parallel work and wait for each other, which would misrepresent the asymmetric nature of grid construction. Using the semantically correct primitive makes the synchronization structure of the system explicit and self-documenting.

- **Per-ball locking and deadlock prevention**: During phase 4, each worker acquires `synchronized` locks on two `Ball` objects before calling `Board.resolveBallCollision()`. To prevent deadlock, locks are always acquired in ascending index order (the index of the ball in the shared `allBalls` snapshot). Because this order is a strict total order and is respected by all workers, no circular wait can occur: a thread holding `lock(ball[i])` and waiting for `lock(ball[j])` with $i < j$ will never be blocked by another thread holding `lock(ball[j])` and waiting for `lock(ball[i])`.

- **ConcurrentHashMap for `lastTouchedBy`**: The `lastTouchedBy` map is accessed from two different contexts: from `WorkerThread`s during phase 4 (to remove entries for colliding balls) and from the `UpdateThread` during phases 5 and 6 under the

`Board` lock (to update and query entries). Using a `ConcurrentHashMap` makes individual operations on the map thread-safe without requiring a `synchronized(this)` block inside `resolveBallCollision()`. This eliminates a potential deadlock that would arise from the lock ordering `synchronized(ball)` → `synchronized(board)`, which could interleave incorrectly with `synchronized(board)` held by `resolvePlayerCollisions()`.

- **BotController monitor**: The `BotController` uses a private `Object botMonitor` as a condition variable. The `BotThread` waits on this monitor inside a guarded loop that checks both `running` and `board.isPlayer2Still()`. The `UpdateThread` calls `botController.signal()` once per tick; if player 4 is still, `signal()` calls `notifyAll()` on the monitor. The `stop()` method calls `notifyAll()` unconditionally, ensuring the `BotThread` is always unblocked on shutdown regardless of the ball's current velocity.

- **ViewModel snapshot and thread safety**: All public methods of `ViewModel` are `synchronized` on the `ViewModel` instance. The `UpdateThread` calls `update()` once per tick, which replaces the internal collections with fresh copies of the model state. The EDT calls getters (`getBalls()`, `getPlayer1()`, etc.) during repaints. Because `update()` returns deep copies (e.g. `new ArrayList<>(balls)`) and all getters return copies as well, the EDT never holds a reference to an object being mutated by the physics threads.

2.1.4 MVC and Multithreading Interaction

The interaction between the MVC layers in the presence of multiple threads is governed by three invariants:

1. **The Model is only mutated by the controller.** The EDT never calls any mutating method on `Board` except `kickPlayer1()`, which is `synchronized`. The `BotThread` calls only `kickPlayer2()`, also `synchronized`. All physics mutations happen exclusively within the sequential and parallel phases orchestrated by `GameController`.

2. **The View never reads the Model directly.** `ViewFrame` and its inner `VisualiserPanel` read exclusively from `ViewModel`. The snapshot is written once per tick by the `UpdateThread` after all physics phases are complete, so the view always paints a consistent state that reflects a completed tick.

3. **Game-over handling is atomic with respect to rendering.** When `checkHolesAndGameOver()` sets `isGameOver = true`, the `UpdateThread` detects this before calling `viewModel.update()` or `view.render()`.

   It calls `handleGameOver()`, which writes the winner message to `ViewModel` via `setGameOverMessage()`, and then renders one final frame. This ensures the game-over overlay is shown on a frame that already reflects the final physical state, with no intermediate partially-updated frame visible to the player.

Figure 2 illustrates the interactions between the active threads of the system during a single game tick. Five participants are represented: the `UpdateThread`, which orchestrates the entire tick; `WorkerThread-0`, which additionally acts as the spatial grid builder;

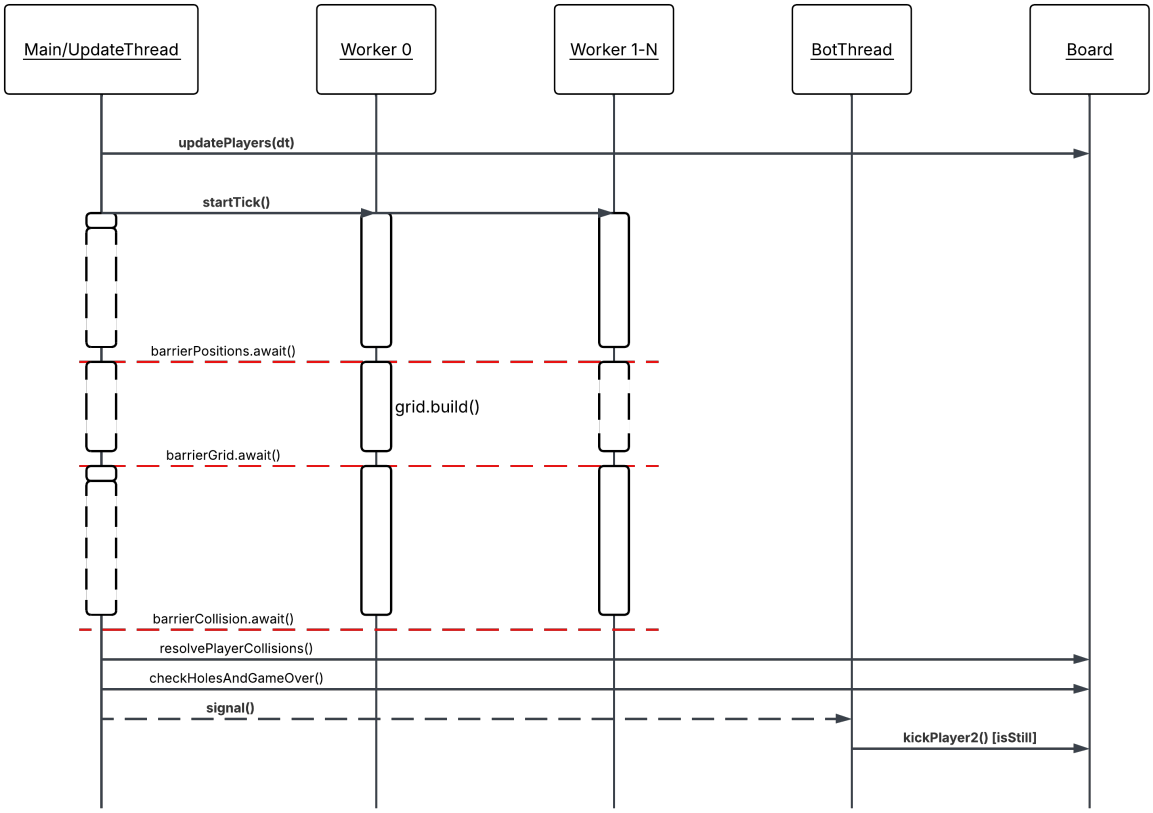

Figure 2: Sequence Diagram of thread interactions and synchronization points during a single game tick.

`WorkerThread-1..N`, representing all remaining worker threads which behave identically; the `BotThread`, which operates independently; and `Board`, included as a passive participant to make explicit which methods are invoked sequentially and by which thread. The Event Dispatch Thread is intentionally omitted, as it interacts with the system exclusively through `synchronized` methods on `Board` and `ViewModel` and does not participate in any of the barrier synchronization points shown. The diagram covers five phases. In **Phase 1**, the `UpdateThread` sequentially calls `board.updatePlayers(dt)` before any parallel work begins. In **Phase 2**, the `UpdateThread` dispatches `startTick()` to all workers and immediately blocks on `barrierPositions.await()`, while `WorkerThread-0` and `WorkerThread-1..N` update the positions of their respective ball partitions in parallel. The first red dashed line marks the firing of `barrierPositions`, after which all threads are guaranteed to have completed position updates. In **Phase 3**, only `WorkerThread-0` performs work — rebuilding the `SpatialGrid` from the updated positions — while all other threads block immediately on `barrierGrid.await()`. The asymmetry in activation box lengths communicates this visually: `WorkerThread-0` has a tall box while the others have near-zero boxes. The second red dashed line marks the firing of `barrierGrid`, establishing the happens-before relationship that allows all workers to safely read the grid in the next phase. In **Phase 4**, all workers resolve ball-ball collisions in parallel using the spatial grid, acquiring per-ball locks in canonical index order to prevent deadlock, while the `UpdateThread` again blocks immediately. The third red dashed line marks the firing of `barrierCollisions`. Finally, in **Phases 5 and 6**, the `UpdateThread` exclu-

sively calls `board.resolvePlayerCollisions()` and `board.checkHolesAndGameOver()` sequentially, then signals the `BotThread` via `signal()` before updating the `ViewModel` and triggering a render.

2.2 Architectural Overview for Task-Based Approach

The task-based concurrent implementation differs fundamentally from the thread-per-entity model. Rather than creating persistent threads for each game component, this approach uses the **Executor Framework** with a fixed-size thread pool to parallelize computation-heavy phases of the game loop. We used **Map-Reduce** algorithms, the architecture implements a Master/Worker pattern where the `GameLoop` acts as the master coordinator. It delegates tasks to a `WorkerPool` abstraction which, knowing the data space topology in advance, splits the workload into smaller, independent chunks (*map phase*). These chunks are then efficiently computed in parallel by the underlying `ExecutorService`.

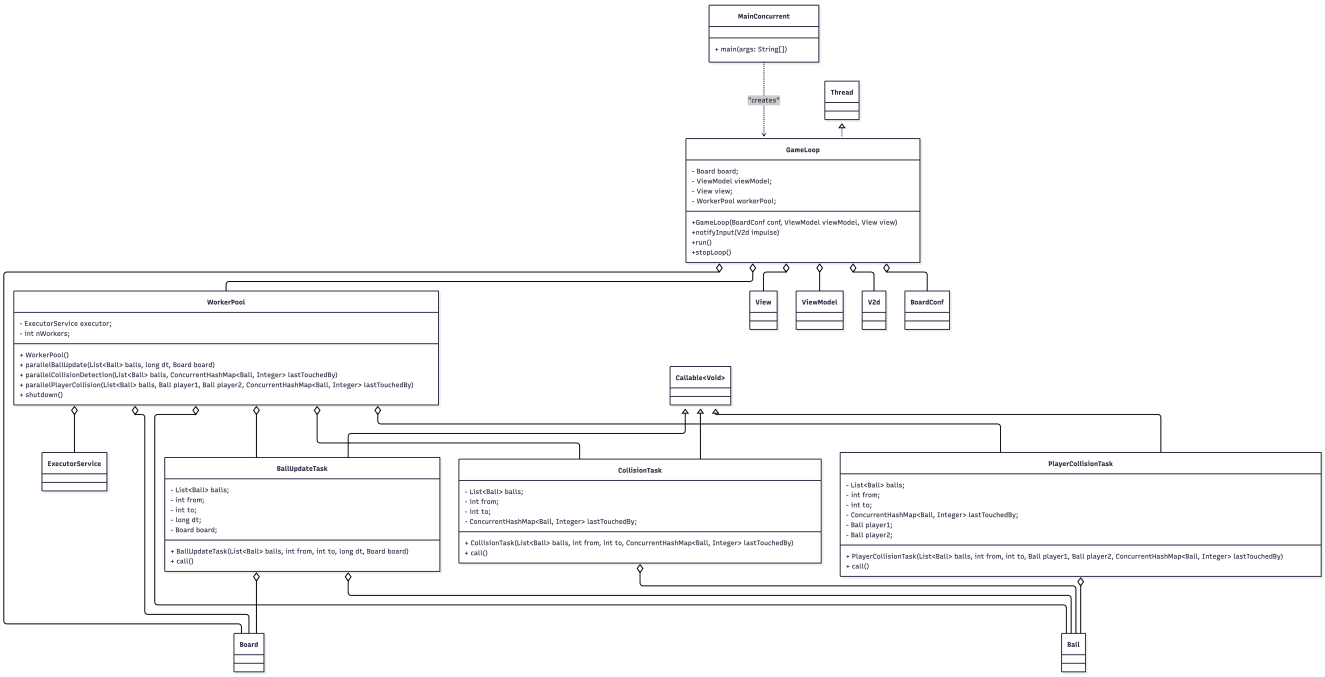

Figure 3: UML class diagram

2.2.1 Component Descriptions

The task-based architecture consists of the following key components:

GameLoop (Master/Orchestrator Thread)

The `GameLoop` extends `Thread` and serves as the main control flow:

- **Role**: Central orchestrator managing game state, input handling, and task delegation
- **Responsibilities**:

- Maintains the main game loop (while !board.isGameOver())
- Manages non-blocking input via `BlockingQueue<V2d> inputQueue`
- Delegates parallelizable phases to WorkerPool
- Executes sequential phase (player/hole collisions) exclusively
- Handles rendering synchronization and frame rate
- Catches `ExecutionException` and `InterruptedException` from worker tasks

- **Key Code Pattern**:

```
while (!board.isGameOver()) {
    V2d impulse = inputQueue.poll(); // Non-blocking input

    // Phase 1: Parallel physics
    workerPool.parallelBallUpdate(board.getBalls(), elapsed,
       board);

    // Sequential player logic
    if (board.getPlayer1() != null)
        board.getPlayer1().updateState(elapsed, board);

    // Phase 2: Parallel collision detection
    workerPool.parallelCollisionDetection(board.getBalls(),
        board.getLastTouchedBy());

    // Phase 3: Parallel player updates
    workerPool.parallelPlayerCollision();

    board.checkHolesAndEndGame();

    // Render
    viewModel.update(board, fps);
    view.render();
}
```

Listing 2: GameLoop Main Loop Structure

WorkerPool (Task Coordinator Abstraction)

The `WorkerPool` encapsulates all ExecutorService framework details from the GameLoop:

- **Role**: Hides complexity of task scheduling, execution, and synchronization
- **Internal Structure**:
  - `ExecutorService executor`: Fixed thread pool with size $k = 2 \times$availableProcessors
  - Three public methods: `parallelBallUpdate()`, `parallelCollisionDetection()`, and `parallelPlayerCollision()`
  - All use `invokeAll()` for barrier synchronization

- **Key Guarantee**: The GameLoop does not import `java.util.concurrent.*`, maintaining clean separation of concerns
- **Exception Propagation**: Exceptions in tasks are bundled into `ExecutionException` and re-thrown at `f.get()`

BallUpdateTask (Parallel Worker)

Implements `Callable<Void>` to update a range of balls' physics state:

- **Input Parameters**:
  - `List<Ball> balls`: Shared reference to all balls
  - `int from, int to`: Range $[from, to)$ of balls this task processes
  - `long dt`: Elapsed time since last frame
  - `Board board`: Context for boundary constraints
- **Computation**:

```
public Void call() {
    for (int i = from; i < to; i++) {
        balls.get(i).updateState(dt, board);
    }
    return null;
}
```

Listing 3: BallUpdateTask Execution

- **Synchronization**: No locks required; each task reads from distinct list indices and modifies only the Ball objects at those indices. The Board's boundary data is assumed immutable during execution.

CollisionTask (Parallel Worker)

Implements `Callable<Void>` to resolve pairwise ball collisions:

- **Input Parameters**:
  - `List<Ball> balls`: Shared reference to all balls
  - `int start, int step`: This task processes indices $start, start + step, start + 2 \cdot step, \ldots$ (interleaved/strided partition, not a contiguous range)
  - `ConcurrentHashMap<Ball, Integer> lastTouchedBy`: Shared map of ball-to-player touches
- **Pair Enumeration**: For each ball $i \in \{start, start + step, \ldots\} \cap [0, n)$, checks pairs (i, j) where $j \in [i + 1, n)$. The strided assignment of i balances the triangular workload across tasks (see *Collision Detection Complexity* above)
- **Synchronization Protocol**:

```
for (int i = start; i < n; i += step) {
    for (int j = i + 1; j < n; j++) {
        Ball a = balls.get(i);
        Ball b = balls.get(j);

        // Ordered lock acquisition (deadlock prevention)
        Ball first = System.identityHashCode(a) <= System.
           identityHashCode(b) ? a : b;
        Ball second = (first == a) ? b : a;

        synchronized (first) {
            synchronized (second) {
                Ball.resolveCollision(a, b);
                // Update lastTouchedBy if collision occurred
                if (velocities changed) {
                    lastTouchedBy.put(a, 0);
                    lastTouchedBy.put(b, 0);
                }
            }
        }
    }
}
```

Listing 4: Ordered Lock Acquisition in CollisionTask

- **Fine-Grained Locking**: Each pair's collision resolution is protected by locks on both Ball objects, but independent pairs may execute concurrently.

PlayerCollisionTask (Parallel Worker)

Implements `Callable<Void>` to resolve player-ball collisions in parallel:

- **Input Parameters**:
  - `List<Ball> balls`: Shared reference to all balls
  - `int from, int to`: Range $[from, to)$ of balls this task processes
  - `Ball player1, player2`: Shared references to *both* player balls — unlike `balls`, these are *not* partitioned: every task instance receives the same `player1`/`player2` objects
  - `ConcurrentHashMap<Ball, Integer> lastTouchedBy`: Shared map of ball-to-player touches
- **Collision Checks**: For each ball $i \in [from, to)$, resolves a collision against `player1` (if present) and against `player2` (if present). This is *not* a pairwise ball-ball enumeration: each ball is only ever checked against the two fixed player objects
- **Synchronization Protocol**: Because `player1` and `player2` are shared by *every* concurrently running task, `Ball.resolveCollision()` on a player must be protected by a lock on that player object, otherwise concurrent tasks could race on the player's position/velocity fields. Each task acquires at most one player lock at a

time (never both together), so this cannot introduce a lock-ordering cycle between `PlayerCollisionTask` instances:

```
for (int i = from; i < to; i++) {
            Ball b = balls.get(i);

            // Collisione con player1 - player1 e' condiviso da tutti i task,
            // il lock serializza gli accessi concorrenti alla sua pos/vel
            if (player1 != null) {
                synchronized (player1) {
                    V2d vPrima = b.getVel();
                    Ball.resolveCollision(player1, b);
                    if (!b.getVel().equals(vPrima)) {
                        lastTouchedBy.put(b, 1);
                    }
                }
            }

            // Collisione con player2 - stesso discorso di player1
            if (player2 != null) {
                synchronized (player2) {
                    V2d vPrima = b.getVel();
                    Ball.resolveCollision(player2, b);
                    if (!b.getVel().equals(vPrima)) {
                        lastTouchedBy.put(b, 2);
                    }
                }
            }
        }
        return null;
```

Listing 5: Per-Player Lock Acquisition in PlayerCollisionTask

Task Decomposition

The `WorkerPool` does not use a single decomposition strategy for all task types: `BallUpdateTask` and `PlayerCollisionTask` use **contiguous range-partitioning**, while `CollisionTask` uses **strided (interleaved) partitioning** to counteract its triangular workload.

- **Contiguous Chunking** (`BallUpdateTask`, `PlayerCollisionTask`):

```
int nWorkers = Runtime.getRuntime().availableProcessors() * 2;
int chunkSize = Math.max(1, n / nWorkers);
for (int i = 0; i < n; i += chunkSize) {
    tasks.add(new BallUpdateTask(balls, i, Math.min(i + chunkSize, n), dt, board));
}
```

Listing 6: Chunk Size Computation in WorkerPool

where n is the number of balls. This ensures:

- If $n = 4500$ balls and $k = 8$ cores (16 workers), chunk size ≈ 281 balls per task
- Minimum chunk size of 1 prevents empty tasks
- Factor of 2 overprovisioning creates more granularity for load balancing
- Suitable here because the per-ball cost is $O(1)$ and uniform across the list, so equal-sized contiguous chunks already balance the load

- **Strided Partitioning (`CollisionTask`)**:

```
int workers = Math.min(nWorkers, Math.max(1, n));
for (int start = 0; start < workers; start++) {
    tasks.add(new CollisionTask(balls, start, workers,
       lastTouchedBy));
}
```

Listing 7: Strided Task Assignment for CollisionTask

Unlike `BallUpdateTask`, the per-ball cost in `CollisionTask` is *not* uniform: ball i is compared against the $n - i - 1$ balls that follow it, so contiguous chunking would give the first task almost all the work and the last task almost none. Assigning index i to task i mod workers instead spreads low- and high-index balls evenly across all w tasks, balancing the total number of comparisons per task.

2.3 Synchronization Strategy

Task-based synchronization employs a layered strategy combining barrier synchronization with fine-grained locking:

2.3.1 Barrier Synchronization via invokeAll()

Parallel phases use `ExecutorService.invokeAll()`, which provides implicit barrier semantics:

- **Behavior**: Blocks until *all* submitted tasks complete
- **Java Memory Model Guarantee**: All field writes in tasks are visible after invokeAll() returns (happens-before relationship)
- **Game Loop Implications**:
  - Phase 1 completes $\implies$ all ball positions/velocities updated and visible
  - Phase 2 completes $\implies$ all collision resolutions applied and visible
  - Phase 3 completes $\implies$ all player updates applied and visible
  - Sequential Phase 4 executes with consistent global state
- **Performance Cost**: Tail latency — the slowest task determines phase duration

2.3.2 Fine-Grained Ball Locking in Collision Detection

CollisionTask uses nested synchronized blocks to prevent data races during collision resolution:

- **Two Locks**: Each `Ball.resolveCollision(a, b)` modifies both Ball objects' velocity and position. Without synchronization, a concurrent update might corrupt the ball state.
- **Deadlock Prevention via Ordering**:
  - All code acquires locks in a consistent order: always lock Ball A before Ball B if identityHashCode(A) $\leq$ identityHashCode(B)
  - Since identity hash codes are fixed per object, this prevents circular wait (requirement for deadlock)
- **Lock Contention**: Multiple tasks may compete for the same Ball locks
  - If two tasks both need to resolve collisions involving the same Ball, one task waits
  - This serializes independent collisions unnecessarily, but ensures safety
  - In practice, contention is moderate since each task processes a different range of "first" balls

2.3.3 Per-Player Locking in PlayerCollisionTask

Unlike `CollisionTask`, where the two colliding balls always belong to the range-disjoint partition and are therefore never touched by two tasks at once, `PlayerCollisionTask` instances all share the *same* `player1` and `player2` objects: every task covers a different range of small balls, but each of them may resolve a collision against either player. Without synchronization, two tasks resolving a collision against `player1` concurrently would race on its `pos`/`vel` fields, silently corrupting the player's state (lost updates, balls appearing to pass through the player). This is fixed by wrapping each `Ball.resolveCollision(player, b)` call in `synchronized(player)`:

- **One Lock at a Time**: A task acquires the lock for `player1`, resolves the collision, releases it, then (independently) acquires the lock for `player2`. It never holds both locks simultaneously, so no lock-ordering scheme is needed to prevent deadlock between `PlayerCollisionTask` instances — a circular wait cannot form when each thread holds at most one of the two contested locks at any time.
- **Contention**: All tasks touching a given player in the same tick serialize on that player's lock; since there are only two player objects, this is a coarser-grained lock than the per-pair locking of `CollisionTask`, but the amount of work performed while holding it (a single `resolveCollision` call) is small.
- **Interaction with the sequential phase**: `Board.checkHolesAndEndGame()` later calls `Ball.resolveCollision(player1, player2)` without any lock, but this happens strictly after `invokeAll().get()` returns for the player-collision phase, i.e. on the GameLoop thread with no worker task running concurrently, so no additional synchronization is required there.

2.3.4 ConcurrentHashMap for Shared State

The `lastTouchedBy` map is a `ConcurrentHashMap<Ball, Integer>` to allow concurrent updates from multiple CollisionTask instances:

- **Concurrent Put Operations**: Multiple tasks call `lastTouchedBy.put(ball, 0)` simultaneously
- **Bucket-Level Locking**: ConcurrentHashMap uses segment-based locking (in modern Java, striped locking)
  - Reduces contention compared to a single global synchronized lock
  - Different Ball keys typically hash to different buckets, allowing concurrent puts
  - Same-bucket updates still serialize
- **Visibility Guarantee**: All puts are visible globally via ConcurrentHashMap's internal synchronization

2.3.5 Exception Handling and Reliability

Task-based execution requires careful exception handling to prevent loop crashes:

- **ExecutionException**: If a task throws an unchecked exception, `f.get()` rethrows it wrapped in ExecutionException
- **InterruptedException**: If the GameLoop thread is interrupted (e.g., window closed), waiting on futures throws InterruptedException
- **GameLoop Protection**:

```
try {
    workerPool.parallelBallUpdate(board.getBalls(), elapsed,
       board);
    workerPool.parallelCollisionDetection(board.getBalls(),
        board.getLastTouchedBy());
    workerPool.parallelPlayerCollision();
} catch (InterruptedException | ExecutionException e) {
    Thread.currentThread().interrupt();
    break; // Exit game loop gracefully
}
```

Listing 8: Exception Handling in GameLoop

- **Graceful Degradation**: Catching exceptions allows the game loop to terminate cleanly rather than crashing the entire JVM

2.3.6 Memory Visibility and Happens-Before Guarantees

The Java Memory Model ensures correctness through explicit synchronization points:

1. **Task Submission**: When a task is submitted to the executor, all field writes in the GameLoop thread are visible to the task

2. **Task Execution**: Workers read shared Ball objects and Board state; without locks, visibility is not guaranteed
3. **invokeAll() Barrier**: When invokeAll() returns, all task writes are visible to the GameLoop thread
4. **Nested Locks in Collisions**: Each `synchronized` block creates a memory barrier, ensuring Ball field updates are visible across tasks

This ensures that despite the distributed nature of tasks, the game state remains consistent across phase boundaries.

3 Behavior Specification: Petri Nets

3.1 Multi-Threaded CyclicBarrier Net

The `CyclicBarrier` is a custom synchronization primitive implemented from scratch. Modelling it as a Petri Net allows its correctness properties — absence of deadlock, liveness, and automatic reset — to be verified independently of the application logic that uses it.

The net represents a generic barrier with $N + 1$ parties, where N is the number of worker threads. Two representative rows are shown explicitly (Thread 0 and Thread N+1), with the intermediate threads indicated by the vertical ellipsis (⋮). Each thread is modelled by three places: *free* (initial state, carries the initial token), *wait* (the thread has called `await()` and is blocked), and *done* (the thread has been released by the barrier and is proceeding). The arc from *done* back to *free* models the cyclic reset: after each tick the thread returns to its initial state, ready for the next barrier cycle.

The single vertical transition *barrier fires* is the core of the net. It has $N+1$ incoming arcs — one from each *wait* place — and $N + 1$ outgoing arcs — one to each *done* place. It is enabled if and only if all $N + 1$ *wait* places simultaneously hold a token, which corresponds exactly to the condition `count == 0` in the Java implementation.

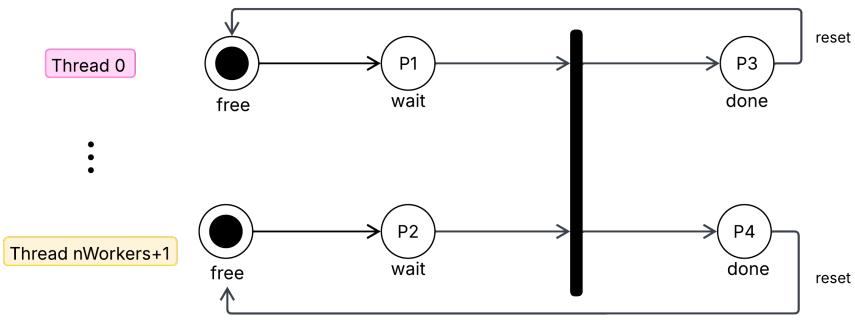

Figure 4: Petri Net modelling the `CyclicBarrier` with $N + 1$ parties. Thread 0 and Thread N+1 are shown explicitly; intermediate threads follow the same structure. The transition *barrier fires* is enabled only when all *wait* places hold a token simultaneously.

The key properties visible from the net structure are the following.

Absence of deadlock. The transition *barrier fires* is enabled if and only if every thread has deposited its token in its *wait* place. Since each thread has no alternative

path that bypasses *wait*, every token will eventually arrive there, guaranteeing that the transition fires. No subset of threads can block the others indefinitely.

Liveness. After *barrier fires*, every *done* place receives a token, releasing all threads simultaneously. The net is live: in any fair execution, every reachable marking eventually leads to the barrier firing and all threads proceeding.

Cyclic reset. The arc from *done* back to *free* restores the initial marking after each cycle, making the barrier reusable across ticks without reallocation. This corresponds to the `generation` counter and the `count` reset in the Java implementation.

Boundedness. The net is 1-safe: at most one token resides in any place at any time. Each thread produces exactly one token per cycle, the transition consumes all tokens atomically, and the loop arc returns exactly one token to *free*. No accumulation is possible.

3.2 Multi-Threaded Tick Flow

The second net models the complete flow of a single game tick across all active threads: the `UpdateThread`, `WorkerThread-0`, `WorkerThread-1..N`, and the `BotThread`. Each thread is represented as a horizontal row of places connected by transitions; vertical transitions that span multiple rows model synchronization points between threads.

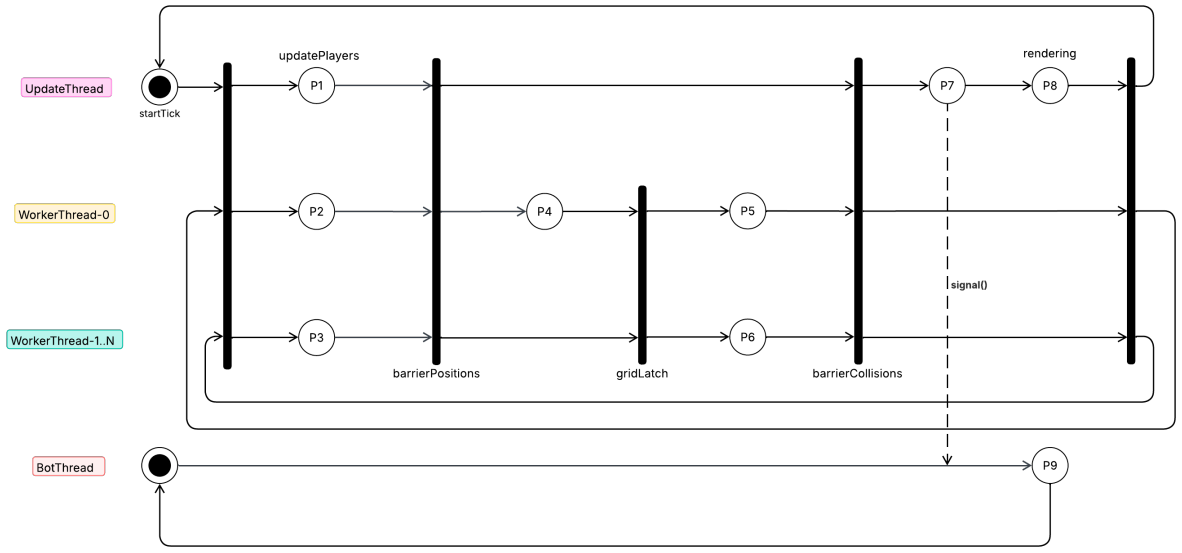

Figure 5: Petri Net modelling the per-thread state flow during a single game tick. Vertical transitions crossing multiple rows represent synchronization points. The `gridLatch` transition spans only the worker rows, reflecting its asymmetric producer-consumer semantics.

Thread states. Each place in the net represents a meaningful state of the corresponding thread. For the `UpdateThread`: *tick start*, *update players*, *await barrierGrid*, *sequential phases*, *rendering*, and *tick done*. For `WorkerThread-0`: *await startTick*, *phase 1*, *grid build*, *phase 2*, and *await next tick*. For `WorkerThread-1..N`: the same sequence except that *grid build* is replaced by a direct arc from `barrierPositions` to `gridLatch`, reflecting the absence of any work during phase G. For the `BotThread`: *await signal* and *kick (if still)*.

Symmetric synchronization — `barrierPositions` and `barrierCollisions`. These two transitions span the rows of all three physics threads (`UpdateThread`, `WorkerThread-0`, and `WorkerThread-1..N`). Each is enabled only when all three rows simultaneously hold a token in the corresponding input place, modelling the all-to-all semantics of the `CyclicBarrier`: every thread must complete its current phase before any thread may advance to the next.

Asymmetric synchronization — `gridLatch`. This transition spans only the two worker rows, not the `UpdateThread` row. Its semantics differ fundamentally from those of the two barriers: it models a one-to-all dependency where `WorkerThread-0` (the producer) opens the latch after `grid.build()` completes, and `WorkerThread-1..N` (the consumers) are unblocked. The `UpdateThread` bypasses the latch entirely, proceeding directly from `barrierPositions` to `barrierCollisions` via its own arc. The asymmetry is visible in the net structure: `WorkerThread-0` has an intermediate place *grid build* before the `gridLatch` transition, while `WorkerThread-1..N` has no such place, reflecting that only Worker-0 performs work during phase G.

BotThread independence. The `BotThread` row is decoupled from the physics rows: it has no shared transitions with the barrier or latch synchronization points. It is connected to the rest of the net only by the dashed `signal()` arc from the `UpdateThread`, which fires after `barrierCollisions` and triggers the bot's kick transition if `isPlayer2Still()` holds. This independence is a deliberate design choice: the bot must not interfere with the physics pipeline timing.

Behavioral properties. The net is 1-safe and live by the same argument as the `CyclicBarrier` net: every thread row has exactly one token in circulation at any time, and every synchronization transition is eventually enabled because no thread has an alternative path that bypasses a barrier or the latch. The loop arcs returning each thread to its initial place after a completed tick confirm that the system can execute indefinitely without reaching a dead marking, consistent with the `no errors detected` result from JPF verification.

3.3 High-Level Game Flow Net Task-based

The system can be broken down into actions called "tasks"; agents are responsible for carrying them out. The computations required to complete each individual task can occur in parallel; however, there are temporal dependencies within the system. The sequential nature of the problem is fundamentally due to the data dependencies between tasks: if a variable must be read in order to complete a task, then it must be ensured that this variable does not change during the read operation, so as not to lose information.

Tasks	Temporal Dependencies	Data Dependencies
Ball Update	Must complete BEFORE Collision Detection	Reads: position, velocity, bounds, elapsed time. Writes: updated position, velocity
Collision Update	Starts AFTER Ball Update barrier; Completes BEFORE Player Collision barrier	Reads: position, velocity, radius, mass (all balls), lastTouchedBy. Writes: velocity, position, lastTouchedBy
Player Collision	Starts AFTER Collision Detection barrier; Completes BEFORE Rendering	Reads: position, velocity, player positions, bounds, lastTouchedBy, scores. Writes: velocity, position, lastTouchedBy, scores, balls list, isGameOver, winnerMessage
Simulation Rendering	Starts AFTER Player Collision; Completes BEFORE next frame input poll	Reads: balls list, player positions, scores, isGameOver, winnerMessage, framePerSec. Writes: ViewModel state, GUI rendering

By following the table, it is possible to represent the basic behavior of the system using a generic Petri net, which is ideally suited for representing the temporal dependencies of individual tasks.

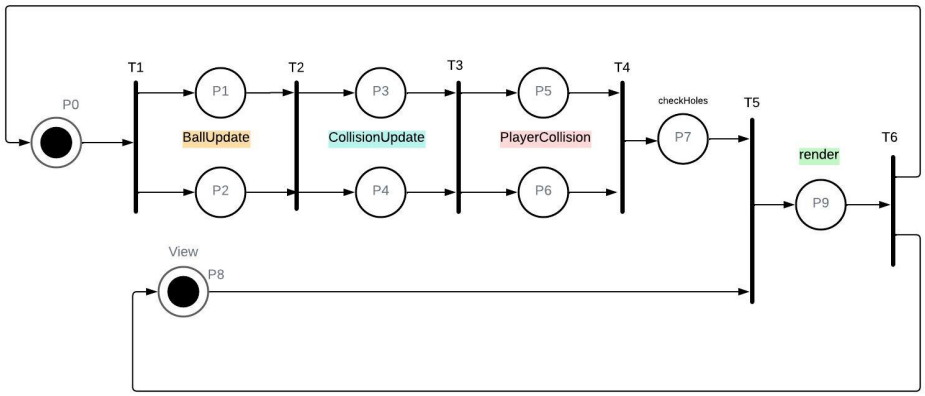

Figure 6: High-Level Game Flow Petri Net

4 Performance Evaluation

This chapter evaluates the runtime performance of the two concurrent implementations of the Poool simulation. Both versions are designed to exploit hardware parallelism to overcome the computational bottlenecks identified in Chapter **??**, but they differ in the concurrency model adopted. Performance is measured in terms of frames per second (FPS), which directly reflects the throughput of the physics pipeline per unit of time.

4.1 Experimental Setup

Hardware

All measurements were collected on a machine equipped with an Intel Core i5-13600K processor (14 cores: 6 P-cores at up to 5.1 GHz and 8 E-cores at up to 3.9 GHz), running the JVM with default heap settings. The number of worker threads is set to `availableProcessors() + 1` at runtime.

Board configurations

Three board configurations are used to stress the system at different scales.

Configuration	Small balls	Naive pairs/tick	Grid pairs/tick (avg)
`MinimalBoardConf`	2	1	1
`LargeBoardConf`	400	79,800	∼2,000
`MassiveBoardConf`	4,500	10,123,750	∼22,500

Table 1: Board configurations used for performance evaluation.

Metric

The primary metric is **frames per second (FPS)**, computed as the running average $fps = \lfloor n_{frames} \cdot 1000/\Delta t \rfloor$ and displayed live in the top-left corner of the window. The frame rate cap introduced by `Thread.sleep()` has been removed, so the loop runs as fast as the hardware allows and FPS reflects raw throughput.

4.2 Multi-Threaded Implementation

The multi-threaded implementation uses a fixed pool of `availableProcessors() + 1` persistent `WorkerThread`s coordinated by two `CyclicBarrier` instances and one `ReusableLatch`. Ball position updates and ball-ball collision resolution execute in parallel across the pool; player collisions and hole detection remain sequential.

Results

Configuration	FPS (i5-13600K)
`LargeBoardConf`	∼400
`MassiveBoardConf`	∼54 – 60

Table 2: Measured FPS for the multi-threaded implementation.

With `LargeBoardConf` (400 balls) the simulation runs at approximately 400 FPS, well above any interactive threshold. The spatial hashing grid reduces the collision workload from ∼79,800 naive pairs to roughly 2,000 candidate pairs per tick, and the parallel pipeline distributes this work efficiently across the available cores.

With `MassiveBoardConf` (4,500 balls) the frame rate stabilises between 54 and 60 FPS, which is acceptable for interactive use. Without the spatial grid this configuration produced only ∼17 FPS in the sequential baseline, confirming that the $O(n)$ algorithmic improvement is the dominant factor: even without parallelism the grid alone would yield

a significant gain, and the worker pool provides the additional multiplier needed to reach interactive framerates.

Bottleneck analysis

At the `MassiveBoardConf` scale the remaining bottleneck is the sequential phase that follows the two barriers: `resolvePlayerCollisions()` and `checkHolesAndGameOver()` both acquire the `Board` lock and iterate over all balls. By Amdahl's law, the non-parallelisable fraction limits the theoretical maximum speedup regardless of how many workers are added. Synchronization overhead at the `CyclicBarrier` and `ReusableLatch` is negligible compared to this sequential fraction at the scales tested.

4.3 Task-Based Implementation

The task-based implementation delegates all parallel work to a `WorkerPool` built on top of `java.util.concurrent.ExecutorService` with a fixed thread pool of `availableProcessors() * 2` threads. Each physics phase is expressed as a collection of `Callable<Void>` tasks submitted via `invokeAll()`, which blocks until all tasks complete before the next phase begins. Three task types are defined: `BallUpdateTask` (position update), `CollisionTask` (ball-ball collision detection), and `PlayerCollisionTask` (player-ball collisions). The `GameLoop` thread acts as master, sequencing the three parallel phases and performing hole detection sequentially after them.

A notable difference from the multi-threaded implementation is that player-ball collisions are also parallelised: each `PlayerCollisionTask` processes a disjoint range of small balls and updates `lastTouchedBy` via a `ConcurrentHashMap`, making the entire phase concurrent. No spatial hashing grid is used: `CollisionTask` tests all pairs within its assigned range against all balls in the full list, preserving the $O(n^2)$ collision detection complexity.

Results

Configuration	FPS (i5-13600K)
`LargeBoardConf`	$\sim$800
`MassiveBoardConf`	$\sim$17 – 18

Table 3: Measured FPS for the task-based implementation.

With `LargeBoardConf` the task-based implementation reaches approximately 800 FPS — twice the throughput of the multi-threaded version at the same configuration. The higher figure is explained by two factors: the thread pool size is `availableProcessors() * 2` (double the worker count of the multi-threaded version), and player-ball collisions are also parallelised, eliminating one sequential phase per tick. The `ExecutorService` framework also introduces less per-tick overhead than explicit `wait/notifyAll` coordination for small workloads.

With `MassiveBoardConf` the frame rate drops to 17 – 18 FPS, essentially matching the sequential baseline. The reason is the absence of the spatial hashing grid: `CollisionTask` still performs $O(n^2)$ pair checks across its range, so the total work per tick remains approximately 10^7 comparisons regardless of how many threads share it. Parallelism

provides a constant-factor speedup, but cannot overcome the quadratic growth of the naive collision algorithm at this scale.

4.4 Comparison

Configuration	Multi-threaded (FPS)	Task-based (FPS)
`LargeBoardConf`	∼400	∼800
`MassiveBoardConf`	∼54 – 60	∼17 – 18

Table 4: FPS comparison between the two implementations on an i5-13600K.

The two implementations exhibit an inverse performance profile across the two configurations, which directly reflects their respective design choices.

With `LargeBoardConf` (400 balls, moderate workload) the task-based implementation is approximately twice as fast. The `ExecutorService` pool uses `availableProcessors() * 2` threads and parallelises all three physics phases including player-ball collisions. The multi-threaded implementation uses fewer threads and keeps player collisions sequential, leaving more work in the critical path per tick.

With `MassiveBoardConf` (4,500 balls, heavy workload) the multi-threaded implementation is approximately three times faster. The spatial hashing grid reduces collision candidates from ∼10 million to ∼22,500 per tick — an algorithmic improvement that no amount of parallelism can replicate without it. The task-based implementation, lacking the grid, saturates all available cores on $O(n^2)$ work and cannot exceed the throughput ceiling imposed by the quadratic complexity.

The comparison demonstrates a fundamental trade-off: the task-based approach achieves higher throughput at moderate scales through a larger thread pool and a fully parallel pipeline, while the multi-threaded approach scales to larger problem sizes through algorithmic optimisation. In a production scenario the two techniques are complementary: the spatial grid would benefit the task-based implementation equally, and the `ExecutorService` model could replace the custom barrier infrastructure of the multi-threaded version.

5 Formal Verification with JPF

This chapter describes the formal verification of the concurrent implementations using Java PathFinder (JPF), a model checker for Java programs developed at NASA. JPF explores all possible thread interleavings of a program under test, checking for violations of safety and liveness properties that may not be exposed by conventional testing.

5.1 Verification Approach

JPF performs exhaustive state-space exploration: rather than executing a single interleaving as a normal JVM would, it systematically branches at every scheduling decision and explores all reachable program states. This makes it capable of detecting concurrency defects — deadlocks, data races, assertion violations — that occur only under specific thread orderings that are unlikely to appear in ordinary test runs.

Because JPF's state space grows exponentially with the number of threads and execution steps, a dedicated entry point `MainJPF` is used instead of the production `Main`. It initialises the board with `MinimalBoardConf` (2 balls), creates only 2 worker threads, and drives the simulation for exactly 3 ticks via explicit `doTick()` calls rather than the unbounded update loop. This bounds the state space while preserving the synchronization structure under test.

`java.util.Random` is not modelled correctly by JPF 8 on Java 11 due to an incompatibility with `jdk.internal.misc.Unsafe`. The `BotController` therefore uses a hand-written LCG generator and is not started in the JPF entry point, since its only interaction with shared state (`board.kickPlayer2()`) is already covered by the `synchronized` method analysis.

5.2 Multi-Threaded Implementation

Properties verified

JPF is configured with the `DeadlockAnalyzer` listener and partial order reduction (`vm.por = true`) to reduce equivalent interleavings. The following properties are checked over all explored states:

Absence of deadlock. No thread scheduling exists under which all active threads block simultaneously waiting for a condition that can never be satisfied. This covers the `CyclicBarrier` (circular wait among parties), the `ReusableLatch` (Worker-0 never failing to call `open()`), and the `tickMonitor` in `WorkerThread` (the `UpdateThread` always calling `startTick()`).

Absence of data races. All accesses to shared mutable state (`Board.balls`, `score1`, `score2`, `isGameOver`, `lastTouchedBy`) occur within `synchronized` blocks or through `ConcurrentHashMap` operations. JPF verifies that no two threads access the same field concurrently without holding the appropriate lock.

Lock ordering correctness. During phase 2, workers acquire locks on two `Ball` objects in strictly ascending index order. JPF explores all interleavings of concurrent `synchronized(a)` and `synchronized(b)` acquisitions and confirms that no circular wait can form.

Results

```
====================================================== results
no errors detected
====================================================== statistics
elapsed time:       00:00:15
states:             new=111913, visited=190331,
                    backtracked=302244, end=0
search:             maxDepth=500, constraints=209
choice generators:  thread=111836
                    (signal=4541, lock=7994,
                     sharedRef=88016, threadApi=2,
                     reschedule=11283), data=0
max memory:         508MB
```

JPF explored 111,913 distinct states and 190,331 visited states within a search depth of 500, completing in 15 seconds without detecting any error. The 111,836 thread choice

generators confirm that the verification covered a large and representative portion of the interleaving space, including all lock acquisitions (`lock=7994`) and shared reference accesses (`sharedRef=88016`).

The `depth limit reached` constraint appearing in the output indicates that some execution paths were truncated at depth 500 rather than reaching a natural termination. This is expected given the cyclic structure of the tick loop: the search depth limit was chosen conservatively to keep verification tractable, and the absence of errors within the explored space provides strong evidence of correctness for the synchronization patterns used.

5.3 Task-Based Implementation

The task-based implementation delegates synchronization to the `java.util.concurrent` framework rather than implementing custom primitives. The `ExecutorService.invokeAll()` call already provides the necessary happens-before guarantee between task completion and the subsequent phase: all submitted `Callable` tasks are guaranteed to have terminated before `invokeAll()` returns, making explicit barrier synchronization unnecessary.

Properties verified

Absence of deadlock. The `CollisionTask` acquires locks on two `Ball` objects using `System.identityHashCode()` as the canonical ordering. Unlike the index-based ordering used in the multi-threaded implementation, identity hash codes are not strictly unique: two distinct objects may share the same code, breaking the total order required for deadlock freedom. JPF would flag this as a potential deadlock source in an adversarial interleaving. In practice the probability of collision is low, but the ordering is not provably safe.

Absence of data races. The `ConcurrentHashMap` for `lastTouchedBy` is accessed concurrently by `CollisionTask` and `PlayerCollisionTask` without additional locking, which is correct since `ConcurrentHashMap` guarantees atomic operations. However, `CollisionTask` resets both `a` and `b` entries to 0 on any velocity change, while `PlayerCollisionTask` writes player indices 1 or 2. Since these two task types run in separate `invokeAll()` phases, no concurrent write conflict arises between them.

Player-object race (identified and fixed). An earlier version of `PlayerCollisionTask` called `Ball.resolveCollision(player1, b)` and `Ball.resolveCollision(player2, b)` with no synchronization at all. Since `player1`/`player2` are shared, unpartitioned objects referenced by *every* task instance in the phase (unlike the balls in `balls`, which are exclusive to one task's range), two tasks resolving a collision against the same player concurrently could race on its `pos`/`vel` fields — a genuine JPF-detectable data race, in contrast to the `lastTouchedBy` accesses above. This was fixed by wrapping each `resolveCollision` call in `synchronized(player1)` / `synchronized(player2)` respectively (§2.3.3). Because a task never holds both player locks at once, this addition does not introduce any deadlock risk symmetric to the one discussed for `CollisionTask` above.

5.4 Discussion

The JPF results confirm that the synchronization design of the multi-threaded implementation is free of deadlocks and data races within the verified state space. The custom

`CyclicBarrier` and `ReusableLatch` primitives behave correctly under all explored interleavings, validating both the implementation and the design choice of using semantically distinct primitives for symmetric and asymmetric synchronization.

The `broken` flag in `CyclicBarrier`, introduced to handle thread interruption without leaving the barrier permanently blocked, was not exercised during JPF verification since no interruptions were injected. Its correctness can be argued structurally from the Petri Net model presented in Chapter **??**: the flag acts as an additional arc that drains all *wait* tokens when an interruption occurs, restoring the net to a state from which no deadlock is reachable.

References

[1] NASA Formal Methods Program, *Java Path Finder - Model Checker for Java Programs*, `https://javapathfinder.github.io/`

[2] J.L. Peterson, *Petri Nets*, *ACM Computing Surveys*, vol. 9, no. 3, 1977.

[3] C.A.R. Hoare, *Monitors: An Operating System Structuring Concept*, *Communications of the ACM*, 1974.

[4] B. Goetz et al., *Java Concurrency in Practice*, Addison-Wesley, 2006.